

操作系统第四阶段 外设和文件

四、文件系统的使用(上)

文件系统的使用过程

- 使用前装载 mount *
 - 将超级块读入内存。分配、释放存储资源需要使用空闲盘块号栈和空闲Inode栈。
 - 将根目录Inode读入内存。搜索文件时要用文件树。
- 使用阶段
 - 搜索、维护文件树 (1) 打开已有文件、目录 (2) 新建文件/子目录时，增加节点和边 (3) 删除文件/子目录时，去除节点和边。
 - 访问普通数据文件。read、write系统调用。写文件会更新DiskInode。
- 使用完毕卸载 unmount *
 - 超级块 和 所有脏数据写回磁盘

文件卷（磁盘）

超级块

inode区

数据区

文件系统的元数据 metadata

- 文件系统持久化在磁盘上的内核数据结构。
- Unix V6++：超级块、DiskInode 和 DirectoryEntry。
- 磁盘上的数据结构，使用前，复制进内存。
- 用完，更新后的数据结构写回磁盘。

文件卷（磁盘）

超级块

inode区

数据区

一、支持文件系统运行的数据结构：系统层面

- 内存Inode表

`g_InodeTable`

每个非空元素是一个内存Inode，管理一个使用中的文件，装有这个文件的DiskInode。

```
class InodeTable
{
public:
    static const int NINODE    = 100;    // 100个内存Inode
public:
    Inode m_Inode[NINODE];           // 内存Inode表
}
```

- 系统打开文件表

`g_OpenFileTable`

每个非空元素是一个File结构，管理一次文件系统会话。

```
class OpenFileTable
{
public:
    static const int NFILE = 100; // 100个File结构
public:
    File m_File[NFILE]; // 系统打开文件表
};
```

内存Inode，访问同一个文件的所有进程共享

class Inode

{

short i_dev;
int i_number;

/* 外存inode所在存储设备的设备号 */
/* 外存inode区中的编号 */

unsigned int i_flag;
int i_count;

/* 状态的标志位，定义见enum INodeFlag */
/* 引用计数 */ open ++, close --

int i_lastr;
static int rablock;

/* 最近读过的逻辑块号，用于判断是否需要预读 */
/* 预读块的物理块号 */

unsigned int i_mode;
int i_nlink;
short i_uid;
short i_gid;
int i_size;
int i_addr[10];

/* 文件工作方式信息 */
/* 文件联结计数，即该文件在目录树中不同路径名的数量 */
/* 文件所有者的用户标识数 */
/* 文件所有者的组标识数 */
/* 文件大小，字节为单位 */
/* 用于文件逻辑块好和物理块好转换的基本索引表 */

};

/* i_flag中标志位 */

enum INodeFlag

{

ILOCK = 0x1,
IUPD = 0x2,
IACC = 0x4,
IMOUNT = 0x8,
IWANT = 0x10,
ITEXT = 0x20

/* 索引节点上锁 */
/* 内存inode被修改过，需要更新 */
/* 内存inode被访问过，需要更新 */
/* 内存inode用于挂载子文件系统 */
/* 有进程正在等待该内存inode */
/* 内存inode对应进程图像的inode */

};

DiskInode复本

DiskInode 的 ID

inode 的使用状态

文件系统会话

- 文件系统会话，是对某个数据文件的一次访问过程。始于open/creat 系统调用，终于close系统调用。
- `fd=open(name,mode);`
 - 访问文件name
 - 文件操作方式是第2个参数mode: FREAD、读; FWRITE、写; FREAD | FWRITE、读写。
- `fd=creat(name,mode);`
 - 创建并打开文件name
 - 文件操作方式是 FWRITE。
- 文件是线性字符流。每个会话有一根读写指针，访问文件时，从这根指针向后读写 n个字节。



File结构管理文件会话，登记目标文件、操作方式和 读写指针

```
class File
{
public:
    enum FileFlags
    {
        FREAD = 0x1,          /* 读操作 */
        FWRITE = 0x2,         /* 写操作 */
        FPIPE = 0x4            /* 管道文件读写 */
    };

public:
    unsigned int    f_flag;    // 文件的操作方式。 FREAD、 FWRITE 或 FREAD | FWRITE
    int             f_count;   // 使用该File结构的进程数量
    Inode*          f_inode;   // 指向目标文件的内存Inode
    int             f_offset;  // 读写指针
};
```

fd = open(name,mode);
fd = creat(name,mode);



二、支持文件系统运行的核心数据结构： 进程层面

- User结构中的打开文件表

```
class User
{
    OpenFiles u_files;
}
```

每个非空元素是进程正在访问的一个文件

```
class OpenFiles
{
public:
    static const int NOFILES = 15;
private:
    File *ProcessOpenFileTable[NOFILES]; // 15个元素
};
```

- 文件描述符 fd

打开文件表的下标，是进程引用文件的内部标识

进程用外部标识（文件路径名）打开文件后，使用内部标识fd读写文件

```
fd = open(name,mode);
fd = creat(name,mode);

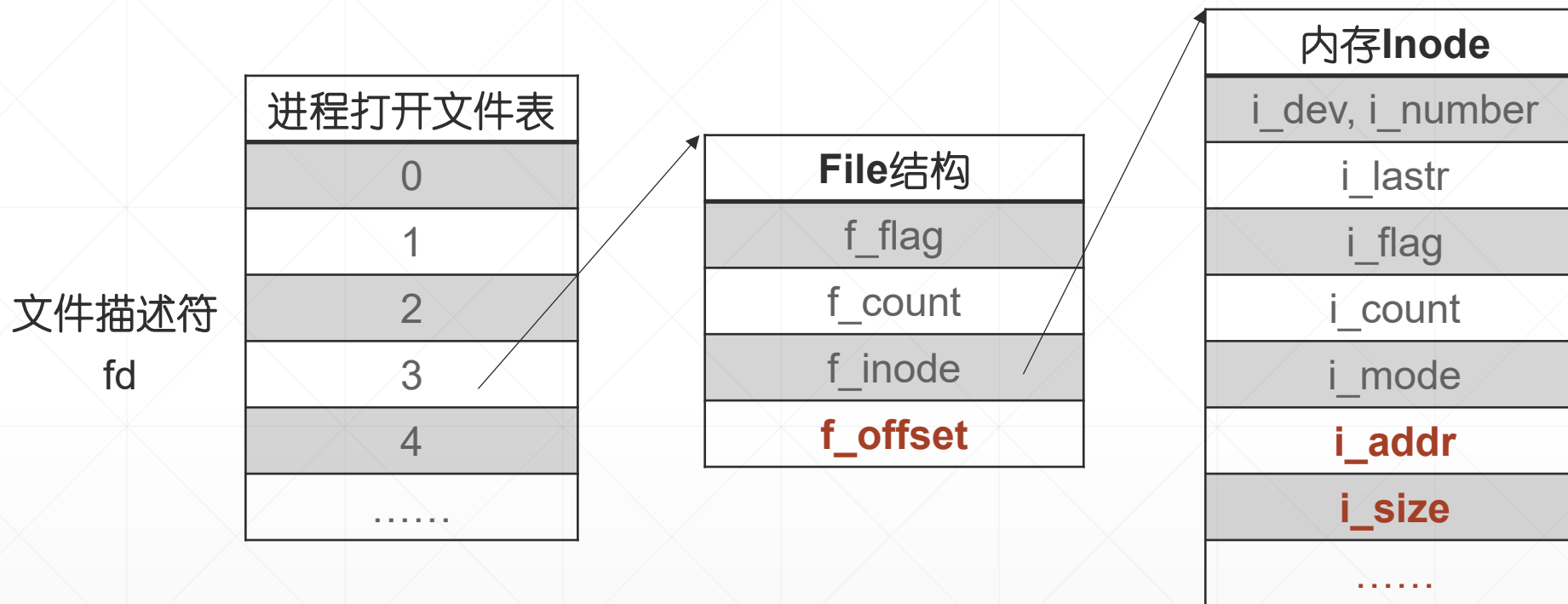
n=read(fd,buf,nbytes);
n=write(fd,buf,nbytes);
seek(fd,offset,ptrname);
```


- 文件IO, 系统调用参数

```
class User
{
    IOParameter u_IOParam;
}
```

```
class IOParameter
{
public:
    unsigned char* m_Base; // 数据块在内存中的首地址
    int m_Offset; // 数据块在文件中的偏移量
    int m_Count; // 数据块尺寸
};
```

三、进程访问数据文件，需要使用打开文件结构



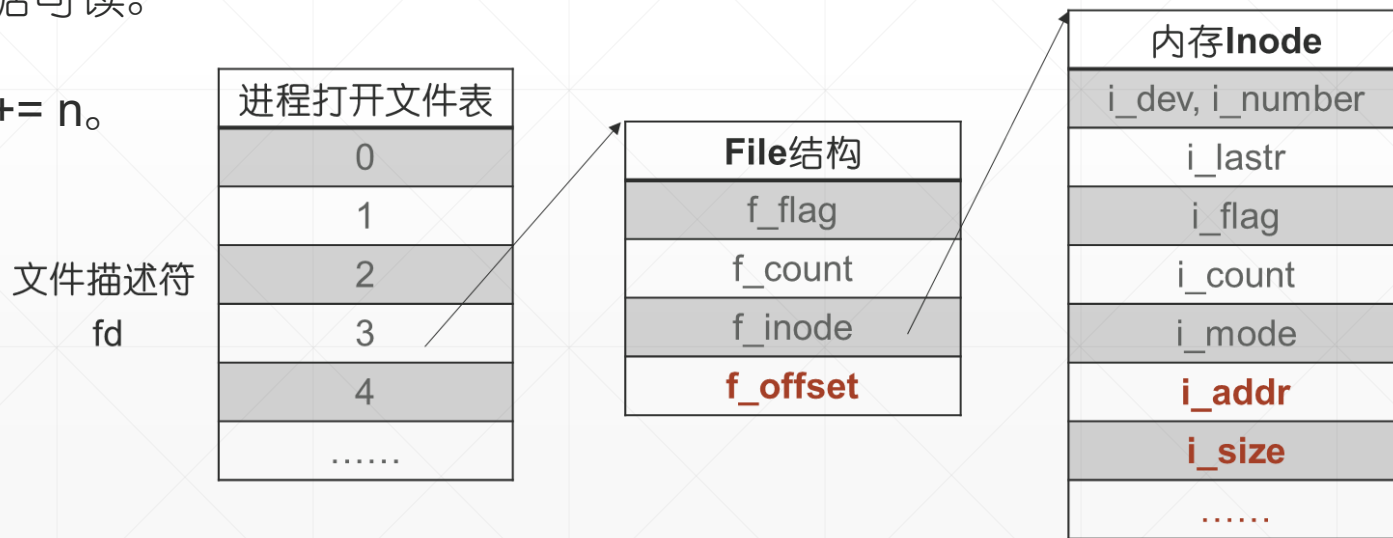
```
n=read(fd,buf,nbytes);  
n=write(fd,buf,nbytes);  
seek(fd,offset,ptrname);
```

1、 $n = \text{read}(fd, buf, nbytes);$

读 fd 引用的文件，从 f_offset 开始，读 $nbytes$ 字节，存入用户空间，首地址为 buf 的内存单元。
返回值 n 是 实际读入的字符数

- (1) $n < nbytes$ ，读至文件尾部。
- (2) $n = 0$ ，EOF (End Of File)，没有数据可读。

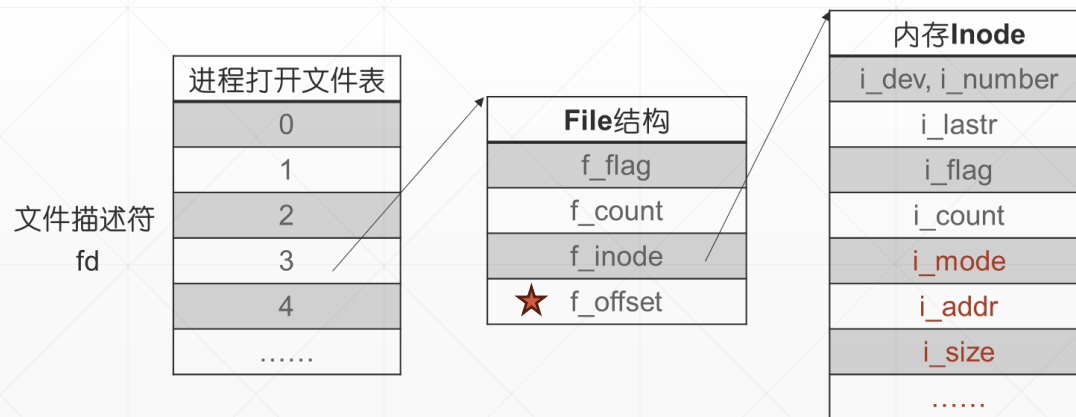
读操作结束后调整文件读写指针， $f_offset += n$ 。



2、 $n = \text{write}(fd, buf, nbytes);$

将用户空间首地址为 `buf` 的一块数据写入 `fd` 引用的文件。从 `f_offset` 开始，向后写 `nbytes` 字节。
返回值 `n` 是 实际读入的字符数，不出意外，一定是 `nbytes`。

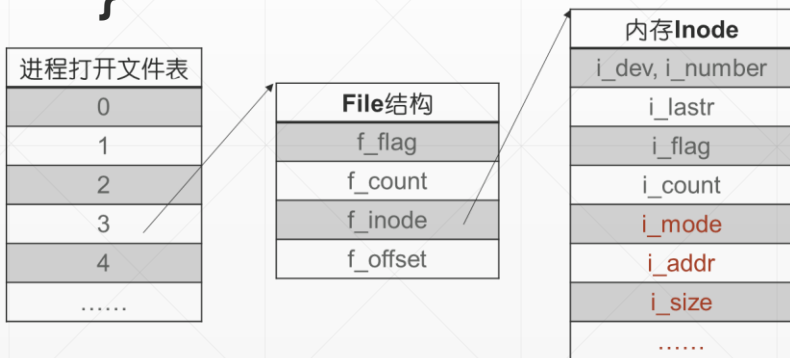
写操作结束后调整文件读写指针，`f_offset += n`。
写操作执行过程中，文件系统为它分配数据块、索引块。
若文件长度增长，更新 `i_size`。



3、read/write 系统调用的实现

```
void FileManager::Read()
{
    this->Rdwr(File::FREAD);
}
```

```
void FileManager::Write()
{
    this->Rdwr(File::FWRITE);
}
```



```
void FileManager::Rdwr( enum File::FileFlags mode )
{
```

```
    File* pFile;
    User& u = Kernel::Instance().GetUser();

    1、检查文件的操作方式
    pFile = u.u_ofiles.GetF(u.u_arg[0]); // fd引用的File结构
    if ( (pFile->f_flag & mode) == 0 ) // FREAD & FWRITE为0, 不可以写读打开的文件
    {
        u.u_error = User::EACCES;
        return;
    }

    2、设置IO参数
    u.u_IOParam.m_Base = (unsigned char *)u.u_arg[1]; /* 用户空间首地址 */
    u.u_IOParam.m_Count = u.u_arg[2]; /* 要求读/写的字节数 */
    u.u_IOParam.m_Offset = pFile->f_offset; /* 数据在文件中的偏移量 */
    u.u_segflg = 0; /* User Space I/O, 读入的内容要送数据段或用户栈段 */

    3、读写文件
    pFile->f_inode->NFlock(); // 锁住内存Inode
    if ( File::FREAD == mode )
        pFile->f_inode->Readl(); // 读文件
    else
        pFile->f_inode->Writel(); // 写文件
    pFile->f_inode->NFrele(); // 内存Inode解锁

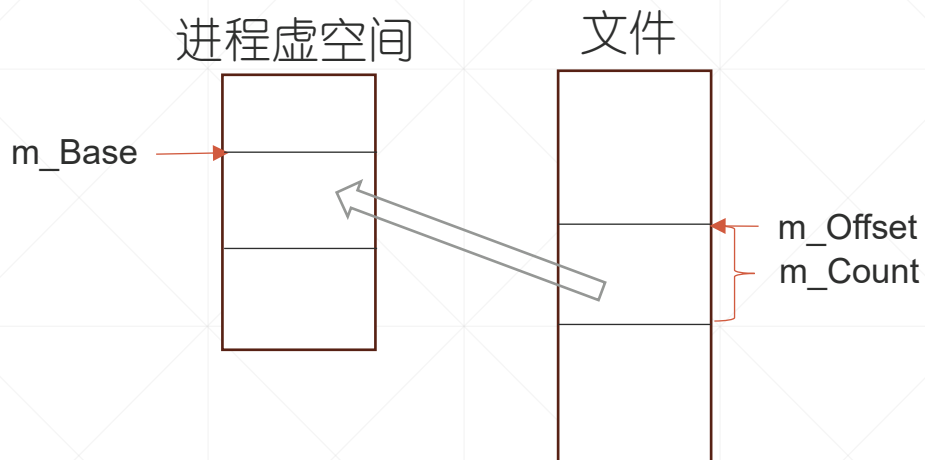
    4、调整文件的读写指针
    pFile->f_offset += (u.u_arg[2] - u.u_IOParam.m_Count); // 实际读写的字符数量

    5、返回实际读写的字符数
    u.u_ar0[User::EAX] = u.u_arg[2] - u.u_IOParam.m_Count;
}
```

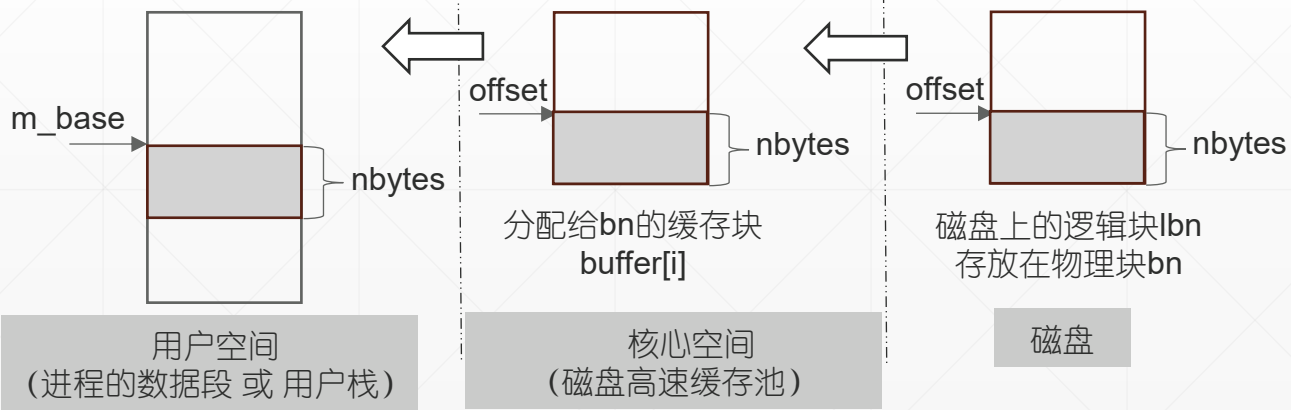
01	1, 读操作合法
10	2, 写操作合法
11	1和2都合法

逐个字符块读写文件
每读写一块, 调整一次IO参数

4、Readl()

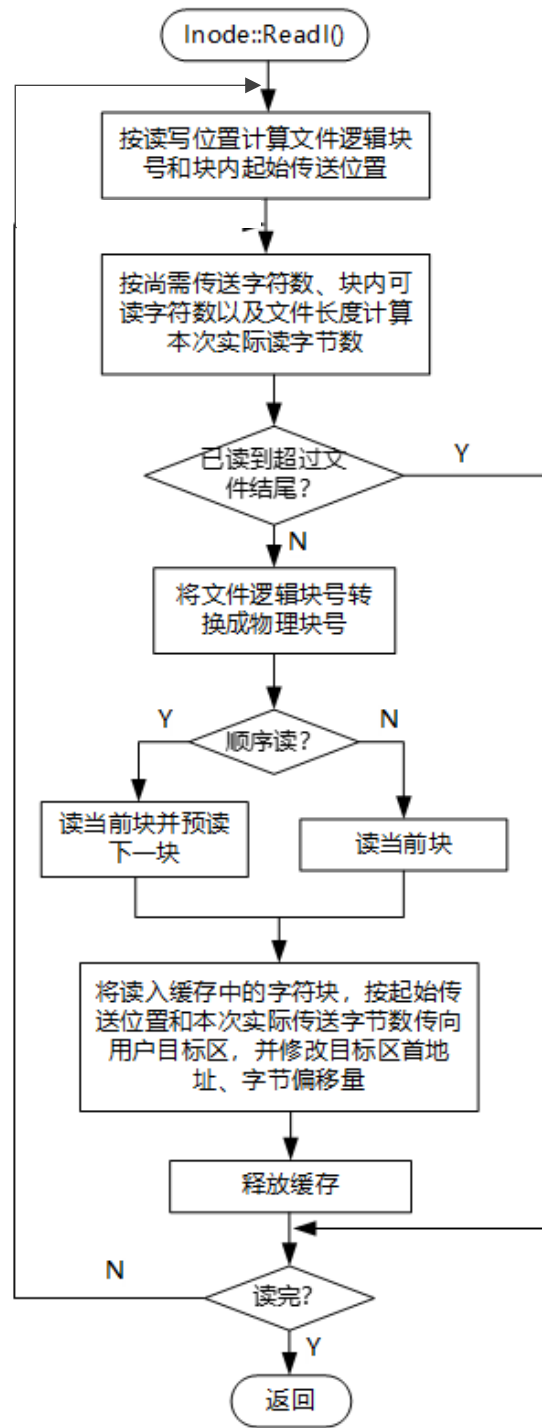


每次读一块（逻辑块号lbn），把这块中需要的数据全读出来（从块内偏移量offset开始，读nbytes字节），送用户空间（m_base）

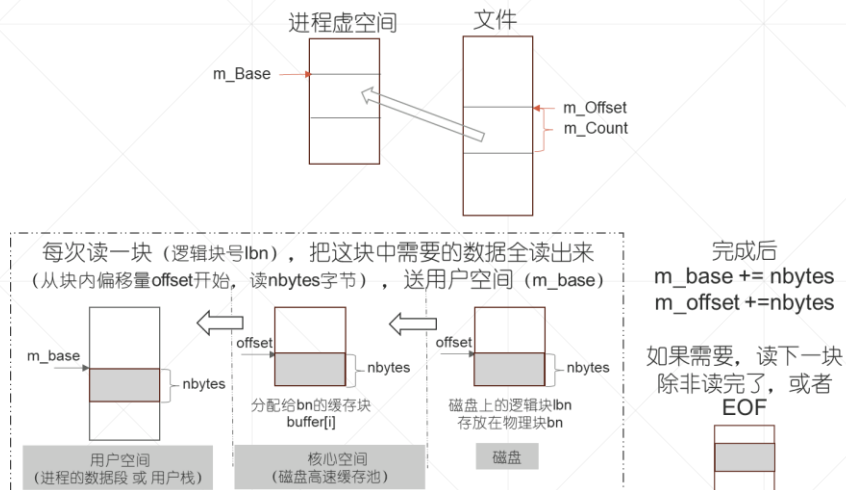


完成后
 $m_base += nbytes$
 $m_offset += nbytes$

如果需要，读下一块
 除非读完了，或者
 EOF



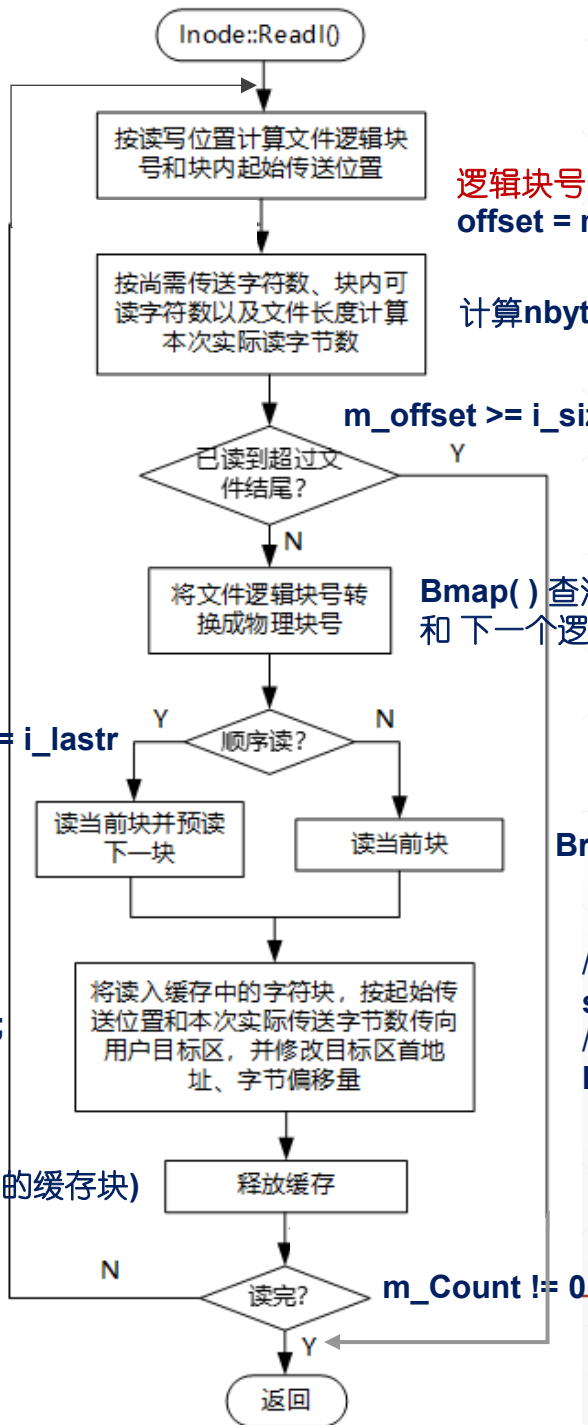
代码注释



Breada(bn, rblkno) (有IO)

u.u_IOParm.m_Base += nbytes;
u.u_IOParm.m_Offset += nbytes;
u.u_IOParm.m_Count -= nbytes;

Brelse(bn的缓存块)



逻辑块号 lbn = m_offset / 512

offset = m_offset % 512 // 这个逻辑块中, 读操作的起始位置

计算nbytes, 这个数据块中需要读入的字节数

m_offset >= i_size (EOF)

Bmap() 查混合索引表计算 lbn 的物理块号 bn
和下一个逻辑块的物理块号 rblkno。

lbn == i_lastr

Bread(bn) (有IO)

// 计算数据在缓存中的首地址
start = pBuf->b_addr + offset;
// 把缓存块中的nbytes字节送至用户区
IOMove(start, m_Base, nbytes);
i_lastr = lbn

m_Count != 0

预读 Breda(bn,rablkno)

- 如果目标文件顺序读，即 $lbn = i_lastr + 1$ ，同步读入当前块时，顺带将下一块 $rablkno$ 异步读入缓存池
 - 将 bn 读入缓存池：
 - 缓存命中？ go
 - 不命中：
 - $GetBlk(rablkno)$ 分配缓存块 $Buffer[i]$
 - $Buf[i]$ 置读标识，送IO请求队列
 - 异步读入 $rablkno$ ：
 - $GetBlk(rablkno)$ 分配缓存块 $Buffer[j]$
 - 缓存命中？
 - Y: $Brelse(\&Buf[j])$ ，解锁
 - N: $Buf[j]$ 置异步标识、读标识，送IO请求队列 (IO完成后，磁盘中断处理程序解锁预读块)
- $Bread(bn)$ 同步等待IO操作结束

预读的好处

- T时刻，进程PA执行如下操作顺序访问磁盘文件A。
文件A总长2个数据块，0#、1#数据块分别存放在50#磁道，扇区号：n、n+2。
执行read系统调用读文件A 0~511字节，访问n#扇区
.....处理读入的新数据（文件A的0~511字节）.....
执行read系统调用读文件A 512~1024字节，访问n+2#扇区
.....处理读入的新数据（文件A的512~1024字节）.....

并发的进程B执行如下操作访问存放在10000#磁道的另一个文件B，读写其中的某个数据块（扇区号m）。
执行read系统调用读文件B，访问 m#扇区

- 磁盘IO使用FCFS调度算法。假设，自由缓存队列有很多不脏的缓存块，n#、n+2#，m#扇区缓存不命中。磁头当前位于0#磁道。PA先执行。问 (1) 采用预读，服务这3个磁盘扇区，磁臂移动总距离



PA执行第1个read系统调用：将同步读请求n 和 异步读请求n+2送入IO请求队列后，入睡等待n#扇区读操作完成。
PA放弃CPU后，PB执行，将IO请求m送入IO请求队列。

上图是这些操作完成后IO请求队列的样子。完成这3个请求，磁臂移动10000-50=9950根磁道。

此外，我们看PA执行第2次read系统调用的细节：n#扇区IO完成后，第1个read系统调用返回。处理完新数据后，PA开始执行第2个read系统调用，它入睡等待n+2#扇区IO完成。。。IO完成后，磁盘中断处理程序解锁 n+2# 数据块，唤醒PA使用预读块数据。

预读块IO几乎没有定位开销（寻道延迟和旋转延迟）。所以，第2次read系统调用，PA睡眠时间很短。

- T时刻, 进程PA执行如下操作顺序访问磁盘文件A。
文件A总长2个数据块, 0#、1#数据块分别存放在50#磁道, 扇区号: n、n+2。

执行read系统调用读文件A 0~511字节, 访问n#扇区
.....处理读入的新数据 (文件A 的0~511字节)
执行read系统调用读文件A 512~1024字节, 访问n+2#扇区
.....处理读入的新数据 (文件A 的512~1024字节)

并发的进程B执行如下操作访问存放在10000#磁道的另一个文件B, 读写其中的某个数据块 (扇区号m)。

执行read系统调用读文件B, 访问 m#扇区

- 磁盘IO使用FCFS调度算法。假设, 自由缓存队列有很多不脏的缓存块, n#、n+2#, m#扇区缓存不命中。磁头当前位于0#磁道。PA先执行。问 (2) 不预读, 服务这3个磁盘扇区, 磁臂移动总距离



PA执行第1个read系统调用: 将同步读请求n 送IO请求队列, 入睡等待n#扇区读操作完成。

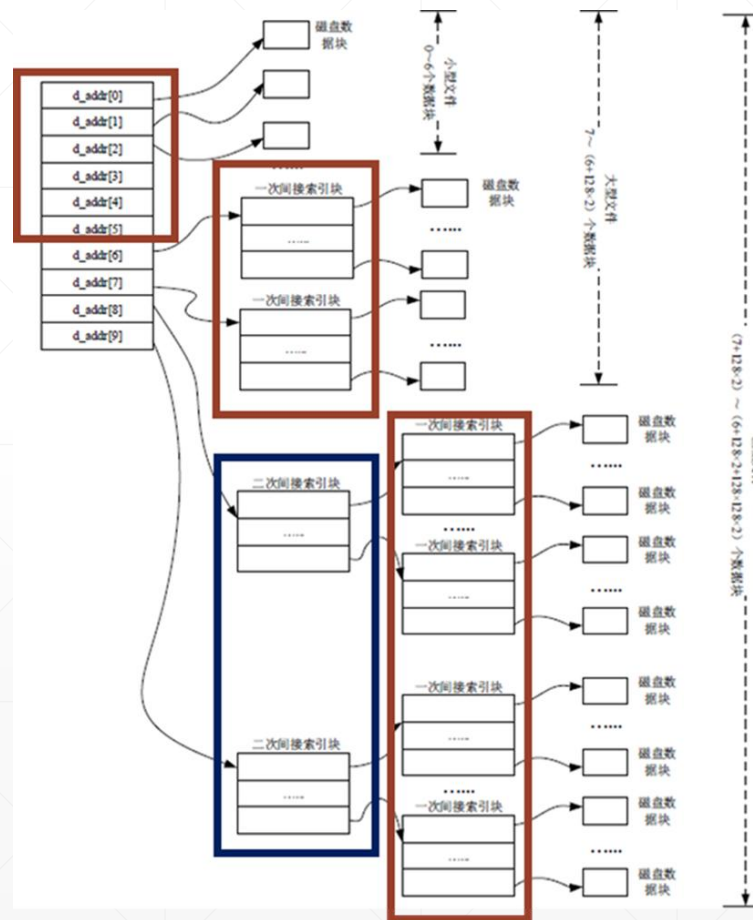
PA放弃CPU后, PB执行, 将IO请求m送入IO请求队列。未来, PA执行第2个read系统调用: 会将同步读请求n+2 送IO请求队列。上图是这些操作完成后IO请求队列的样子。完成这3个请求, 磁臂移动 $2 * (10000 - 50)$ 根磁道。

~~相邻扇区IO请求没法挨在一起, 整个磁臂移动轨迹有很多折返, 磁盘吞吐率下降了。~~

总结, 预读的好处: 在进程执行顺序读操作时, 预读可以减小磁臂移动总长度, 缩短读操作耗时。
前提: 文件系统会尽量为相邻逻辑块分配相邻的物理块。

顺序读操作，何时放弃预读？

- 同步块bn缓存命中时，构造预读块，价值大嘛？
- 如果计算预读块的物理块号rablkn需要读入索引块，可以考虑放弃预读。



open后，Inode常驻内存。
i_addr[0] ~ i_addr[5]在内存里，访问无需IO。
其余物理块号，在磁盘上。访问前需要IO。

假设，文件A所有逻辑块缓存不命中。不考虑预读，请问：

- 1、读偏移量600的字节，几次IO？
- 2、随后，读偏移量1000的字节，几次IO？
- 3、随后，读偏移量3100（6#逻辑块，28#字节）的字节，几次IO？

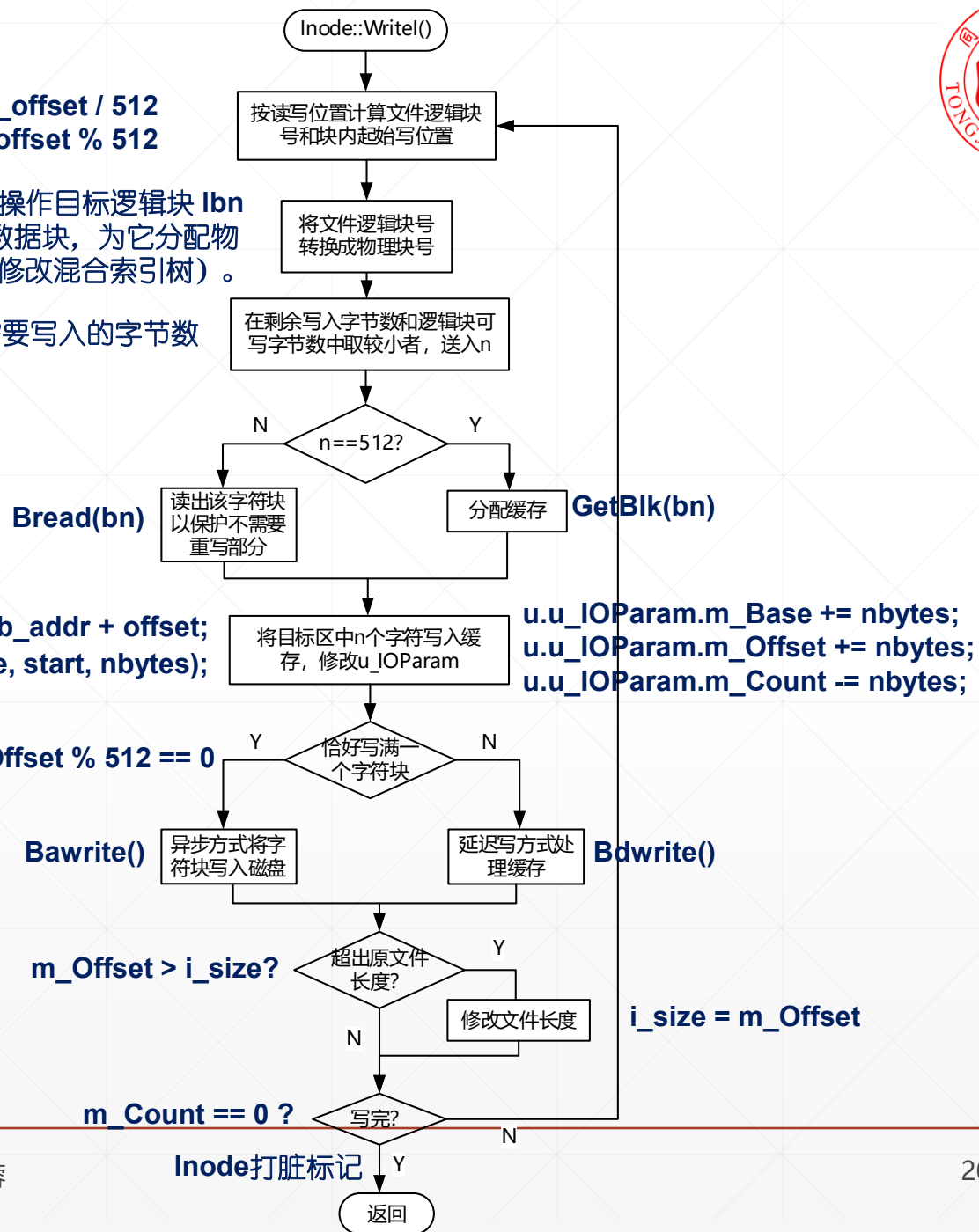
考虑预读，顺序读文件A，读 lbn==5 的逻辑块时，有必要预读吗？

5、Writel()

逻辑块号 $lbn = m_offset / 512$
 $offset = m_offset \% 512$

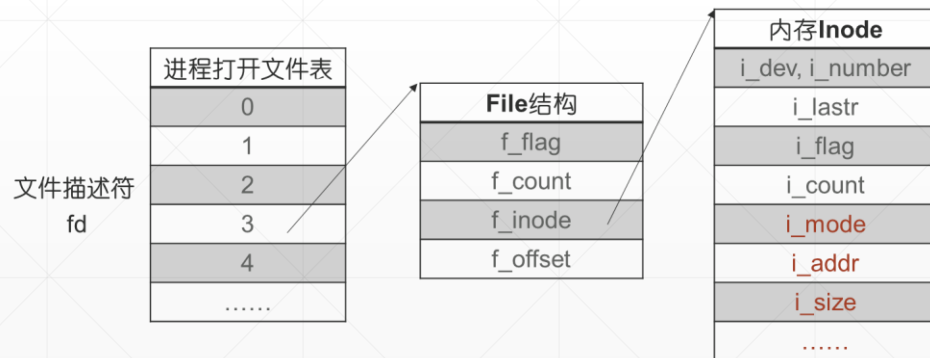
Bmap() 查混合索引表计算写操作目标逻辑块 lbn 的物理块号 bn 。若 lbn 是新数据块，为它分配物理块，必要时，分配索引块（修改混合索引树）。

计算 n ，数据块中需要写入的字节数



6、seek()

`seek(fd,offset,ptrname);` // 调整文件读写指针



```
void FileManager::Seek()
{
    .....
    int fd = u.u_arg[0];    // fd

    pFile = u.u_ofiles.GetF(fd); // File结构

    .....
    int offset = u.u_arg[1]; // offset

    .....
    switch ( u.u_arg[2] ) // ptrname
    {
        /* 读写位置设置为offset */
        case 0:
            pFile->f_offset = offset;
            break;

        .....
    }
}
```

习题 1

一、完整的文件读写过程

1、

```
int fd = open("/usr/ast/Jerry", 3); //以可读可写方式打开文件
char data[300];
seek(fd, 500, 0); //将文件读写指针定位到第500字节
int count = read (fd, data, 300); //从文件读300字节到data
count = write(fd, data, 300); //从 data 写 300 字节到文件
```

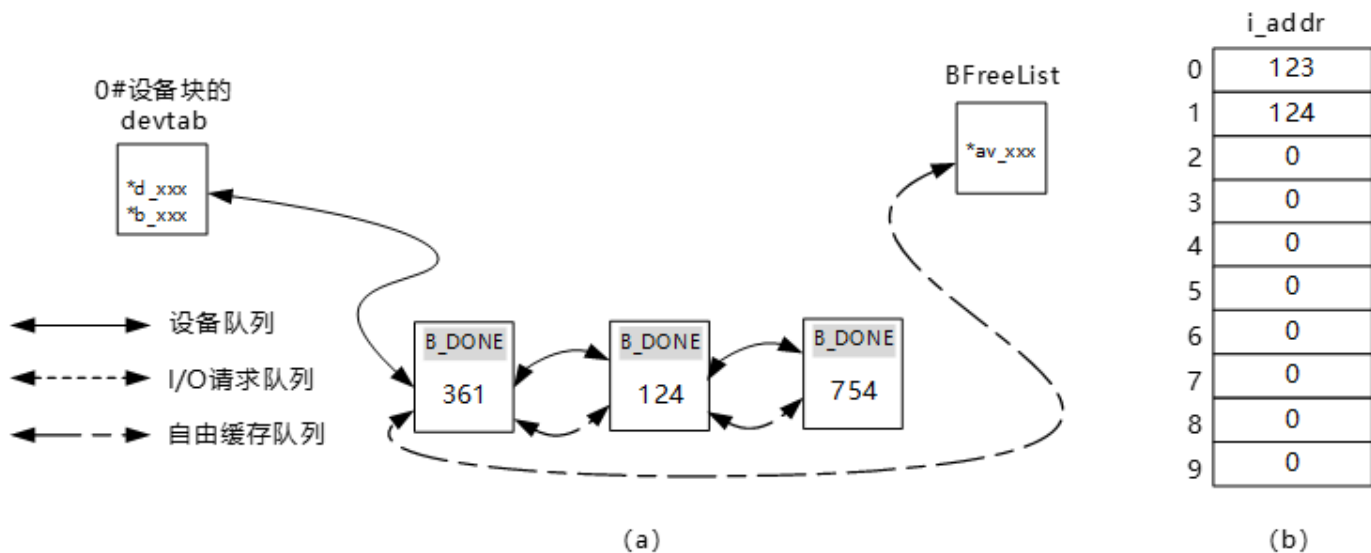


图 1、缓存队列 和 Jerry 文件的索引表

习题 2:

3、open 系统调用结束后，读下图中的文件 file1，假设所有数据块和缓存块缓存不命中，不考虑预读。进程依次执行以下读操作，回答问题（1）读 3#逻辑块，几次 IO？（2）读 6#逻辑块，几次 IO？（3）读入 6#逻辑块之后，接着读 100#逻辑块，几次 IO？（4）随后，写 198#逻辑块中的第 200#字节，几次 IO？

[参考答案]

open 系统调用结束后，file1 的 Inode 在内存里。

（1）读 3#逻辑块，1 次 IO。0#~5#逻辑块的物理块号在 Inode 中，文件打开后在内存。所以，读 3#逻辑块 IO 一次，读入物理块 d3。

（2）读 6#逻辑块，2 次 IO。第一次读入物理块 i0、得到文件的第一个索引块，get 物理块号 d6。第二次读入物理块 d6，get 文件数据。

（3）读 100#逻辑块，1 次 IO。100#逻辑块的物理块号登记在第一个索引块，已经在内存里了。所以，只需一次 IO 读入物理块 d100。

（4）写 198#逻辑块中的第 200#字节，2 次 IO。一次读入物理块 i1，获得第 2 个索引块，查询得知 198#逻辑块存放在 d198#物理块中。第二次 IO，执行先读操作，将物理块 198 读入磁盘高速缓存。没有写 IO，因为这个逻辑块没有写满，就让它呆在缓存池里。

